

Lecture 01 Handout — The Note-Set Starter: One Specification and Three Process Documents

Software-engineering module, Lecture 01. Read before the first meeting (ten minutes). Keep it open during the demo and while doing Exercise 1.

This handout reproduces the files that the Lecture 01 demo repository contains at the start of class, before any note exists. They are copied from `demos/lecture-01-note-specs-demo/starter/` in the course repository; only the heading levels are shifted so that the handout has one title.

File	Role in the lecture
<code>note-format-spec.md</code>	The specification: six numbered rules, R1 through R6, that every note must satisfy. Version 0.1, a draft, and it contains defects the demo's audit finds.
<code>CLAUDE.md</code>	The loader: names the governing document and the process documents; two rules for the agent.
<code>process/spec-audit.md</code>	How a specification's quality is assessed — the five quality properties as rules AUD-1 to AUD-5, where to look (AUD-6), and what the audit produces (AUD-7). Run when asked.
<code>process/reporting.md</code>	The forms of a report: a gap list (RPT-1), a conformance report (RPT-2), an amendment proposal (RPT-3), and the rule that nothing is repaired before a ruling (RPT-4). Always loaded.
<code>process/verification.md</code>	The three kinds of verifier and the scope of a result (VER-1); mechanical and judgment clauses (VER-2). Always loaded.

There is no concept of operations in this starter; it arrives in Lecture 02. During the demo the specification is amended to version 1.0.0 (a scope statement, an extended R3, a new R7); the versions here are the starting point. Exercise 1 starts from a copy of these files.

File: `note-format-spec.md` — the specification.

Note Format Specification

Version: 0.1 (draft) Status: draft for review; not yet realized.

This is a normative specification. The keywords **MUST**, **MUST NOT**, **SHOULD**, and **MAY** carry their RFC-2119 meanings. Rules are numbered so that conformance reports — whether produced by an agent or by a tool — can cite them. Every markdown file in the note set **MUST** satisfy the rules below.

Rules

R1 — Front-matter block Every note **MUST** begin with a YAML front-matter block delimited by `---` lines. The opening `---` **MUST** be the first line of the file, and a closing `---` **MUST** follow the front-matter fields.

R2 — Front-matter fields The front-matter **MUST** contain exactly two fields:

- `title` — a non-empty string.
- `created` — the note's creation date in ISO-8601 form (YYYY-MM-DD).

R3 — Title heading The note body MUST begin with a level-1 heading (#), which MUST be the first non-blank line after the front-matter.

R4 — Single H1 A note MUST contain exactly one level-1 heading.

R5 — No skipped heading levels A heading MUST be at most one level deeper than the nearest heading above it. A ## section MAY open ### subsections, but a ## heading MUST NOT be followed directly by a #### heading.

R6 — Index consistency Every note file MUST be linked from `index.md` exactly once, and every note link in `index.md` MUST resolve to an existing note file.

File: `CLAUDE.md` — the loader.

Note set — agent orientation

What this repository is

A set of markdown study notes maintained by an agent that follows a written specification. At the start there are no notes: there is a specification, and there is a process for working with the specification.

Governing documents, in authority order

1. `note-format-spec.md` — what a conformant note is: rules R1–R6.

Process — how work proceeds with respect to the specification

The rules in the following documents are always in effect (always on) (loaded with this file): `@process/reporting.md` `@process/verification.md`

The rules in the following document are only invoked when needed (On demand) — run before any plan, after any changes to the specification, or when asked: `process/spec-audit.md` (AUD).

Bootstrap rules

1. Read the governing documents and the always-on process documents, in full, before any operation.
2. Never modify a governing document or a process document without explicit approval; propose changes as an amendment proposal (RPT-3) and wait.

Commands

- `python3 check_notes.py --all` — the algorithmic verifier (VER-1), once it exists.

File: `process/spec-audit.md` — how a specification is audited (AUD).

Specification audit (AUD)

Version: 1.0 Status: normative; maintained. **Invoked on demand** — not loaded every session.

This document specifies how the quality of a specification is assessed before the specification is relied on. Any kind of verifier — human, agent, or algorithmic — may perform the audit, and the output has the same form whoever produces it (RPT-1). The keywords MUST, MUST NOT, SHOULD, and MAY carry their RFC 2119 meanings. Rules are numbered AUD-1... so that audit reports, rulings, and later documents can cite them.

When the audit runs

- When the developer asks for it (“run the specification audit on X”).
- Before any plan is proposed that would realize the specification.
- After any amendment to a specification — on the amended document, and on every document that cites it.

In this repository

Under audit	Realizations governed	Clauses cited as
<code>note-format-spec.md</code>	the files in <code>notes/</code>	R1, R2, ...

Rules

AUD-1 — Unambiguous Every clause **MUST** decide yes or no for any realization put in front of it. Requirement levels **MUST** use the RFC 2119 keywords. Wherever a clause constrains form — a date, a filename, a layout — it **MUST** include an example that is exact to the character.

Test: for each clause, could two careful readers disagree about whether a given realization satisfies it? If so, the clause is ambiguous.

Example: “the `created` field holds a date” is ambiguous; “an ISO-8601 date, `YYYY-MM-DD`” is not.

AUD-2 — Internally consistent No two clauses of one document **MUST** conflict, and any two clauses that constrain the same thing **MUST** state their relationship.

Test: list every pair of clauses that touch the same property and write the relationship down. A pair with no stated relationship is a finding (i.e., a potential problem with the specification).

Example: R2 requires a `title` field and R3 requires an H1; nothing says they agree. Two clauses, one property, no relationship.

AUD-3 — Externally consistent and aligned Across the specifications of one system: one vocabulary (a glossary), no requirement in one document that an artifact of another document cannot satisfy, and every fact stated once or stated identically.

Test: for each artifact any document names, check every document’s rules against it; for each term, check that it carries one meaning everywhere.

Example: a format specification that governs “every markdown file in the note set” and a second document that requires an `index.md` with no front-matter: no `index.md` can satisfy R1. With a single document in the repository this rule has nothing to compare; it becomes active when a second document arrives.

AUD-4 — Complete for its level Completeness is judged at the specification’s own level of abstraction, not against everything that could be said. Every mode, policy, input form, and output form that a higher-level document names **MUST** be addressed, and every operation the specification implies **MUST** be performable without inventing a convention.

Test — the execution walkthrough: perform each operation step by step, on paper, and list every point at which you had to invent something.

Example: “add a blank note” takes a title; no clause says what filename the note gets. The walkthrough cannot proceed without inventing one.

AUD-5 — Traceable Every clause MUST carry a stable identifier. Identifiers MUST NOT be renumbered when a specification changes; new clauses are appended. A document derived from a specification MUST cite the clauses it derives from, and a verification result MUST be able to cite the clause it decides.

Test: pick a clause. Can a report, a test, or a derived document name it unambiguously — and will that name still be right after the next amendment?

AUD-6 — Search in three places An audit MUST look (a) within each document, (b) between documents, and (c) between a document and the act of executing it. The report MUST state which of the three were searched; a gap list that searched one place is incomplete.

AUD-7 — Output, and stop The audit’s output is a gap list in the form RPT-1: numbered, each item placed, categorized, quoted, and given a recommended resolution. The auditor MUST then stop and wait for rulings. The auditor MUST NOT amend a specification or begin a plan on the strength of its own findings.

A question that can be answered by reading the repository is answered by reading the repository, not by asking.

Changelog

- **1.0** (2026-09-19) — Initial: AUD-1 through AUD-7.

File: `process/reporting.md` — the forms of a report (RPT).

Reporting (RPT)

Version: 1.0 Status: normative; maintained. **Always on.**

This document specifies the forms in which findings, results, and proposed changes are reported, so that a person can act on them and so that the repository’s history records what was found, what was decided, and why. A report is evidence with a scope; the forms below exist so that the scope is always stated. The keywords MUST, MUST NOT, SHOULD, and MAY carry their RFC 2119 meanings.

In this repository

Reports cite `note-format-spec.md` clauses as R1, R2, ... and process rules as AUD-n, VER-n, RPT-n. Reports are delivered in the conversation; a report that leads to a change is summarized in that change’s commit message.

Rules

RPT-1 — Gap list (the output of an audit) A numbered list. Each item MUST give:

- **Where** — document and clause or section, or “absent”.
- **Category** — one of: ambiguous · incomplete · inconsistent within a document · inconsistent between documents · execution gap (a step that cannot be performed without inventing something).
- **Quoted text** — the clause as written, or the text that should exist and does not.
- **Recommended resolution** — one concrete proposal.
- **Status** — open · ruled: *the ruling* · deferred.

Items are ordered by the document they concern. The list MUST end by stating which of the three search places (AUD-6) were covered.

Example:

3. **Where:** `note-format-spec.md`, R2 and R3. **Category:** inconsistent within. **Quoted:** R2 “`title` — a non-empty string”; R3 “The note body MUST begin with a level-1 heading”. **Recommended:** amend R3 to require the H1 text to equal `title`, with the field authoritative. **Status:** open.

RPT-2 — Conformance report (the output of a verification) Before any finding, a conformance report MUST state:

- **Verifier** — which kind (human, agent, or algorithmic) and which one, with a version: the script’s commit; the model and date for an agent; the reader’s name for a person.
- **Specification and version** — the document, and the version it was checked against.
- **Clauses checked** and **clauses not checked**, by identifier.

Then, for each finding: the clause identifier; the clause text, quoted; the location in the realization; what was observed. Then a verdict: *conformant*, or the list of violated clauses. Repairs MAY be proposed; none are applied (RPT-4).

The three scope lines exist because a pass against a stale version, a verifier that decides only some clauses, and a result that does not say which verifier produced it are the three ways a true-looking result misleads.

Example:

Verifier: agent (Claude Code, 2026-09-11). **Specification:** `note-format-spec.md` 1.0.0. **Checked:** R1–R7. **Not checked:** none. **R2** — “`created` — the note’s creation date in ISO-8601 form (YYYY-MM-DD)”: front-matter line 3 reads `created: Sept 11, 2026`. **R3** — “The H1 text MUST be identical to the front-matter `title`”: the H1 is `The Waterfall Paper`; `title` is `Royce 1970 Waterfall Paper`. **Verdict:** nonconformant (R2, R3, R4, R5). Repairs proposed below; none applied.

RPT-3 — Amendment proposal (a change to a specification or a process document) A numbered list. Each amendment MUST give: the document; the clause, or “new clause” with its identifier; the old text and the new text; the rationale, citing the gap-list item or finding that motivates it; the version bump the whole proposal implies (patch, minor, or breaking); and the changelog entry that will record it. The proposal MUST be marked *awaiting approval*, and nothing MUST be edited until approval is given.

RPT-4 — Report before repair When a finding is made — by an audit or by a verification — the report MUST be delivered and read before any realization or specification is changed on its account. Which side moves is the developer’s decision. The decision and its reason are recorded in the changelog entry when the specification moves, and in the commit message when the realization moves.

Changelog

- **1.0** (2026-09-19) — Initial: RPT-1 through RPT-4.

File: `process/verification.md` — how conformance is verified (VER).

Verification (VER)

Version: 1.0 Status: normative; maintained. **Always on.**

This document specifies how conformance between a realization and its specification is checked: by which kinds of verifier, with what scope, and which clauses each kind may decide. Later versions add when checks must run and what gate a change must pass. The keywords MUST, MUST NOT, SHOULD, and MAY carry their RFC 2119 meanings.

In this repository

Realization	Specification	Verifiers available
a note in <code>notes/</code>	<code>note-format-spec.md</code>	a person reading both; the agent, asked to check; <code>check_notes.py</code> , once it exists

Rules

VER-1 — Verifiers and scope Conformance MAY be verified (a) by a **human verifier** — a person reading the specification and the realization and manually checking the correspondence between the two (b) by an **agent verifier** — an agent asked to check and report — or (c) by an **algorithmic verifier** — a program written to decide the specification’s clauses. These are the three kinds of verifier. Every result MUST state which verifier produced it and MUST be reported in the form RPT-2, so that its scope — which verifier, which version of the specification, which clauses — is explicit.

	Human	Agent	Algorithmic
Cost per check	minutes of attention	tokens	milliseconds
Same result on repeat	not guaranteed	not guaranteed	yes
Decides mechanical clauses	yes	yes	yes
Decides judgment clauses	yes	yes	no
Composable into a gate	no	with effort	yes, via the exit code

When verification is performed, this doesn’t give us a “proof” that the realization meets the specification. It only provides some “evidence within a given scope” that the verification holds. Unless we formally prove the verifier correct, any verifier can be wrong: it may implement or apply a clause incorrectly, run against a stale version of a specification, judge from memory instead of from the document, or decide fewer clauses than its output implies. The reporting instructions of RPT-2 include reporting on the scope so that we can see possible gaps.

VER-2 — Mechanical clauses and judgment clauses Every clause of a specification MUST be classified as **mechanical** — decidable by a program from the artifact alone — or **judgment** — requiring a reader’s assessment (“accurately summarizes”, “preserving the user’s meaning”). The classification is kept in the table below and MUST be amended whenever the specification changes. An algorithmic verifier decides mechanical clauses only; judgment clauses are named as remaining for a human or an agent verifier, never silently assumed to be covered.

Clause	Algorithmic	Human or agent
R1 front-matter block	yes	—
R2 fields, including date validity	yes	—
R3 H1 placement	yes	—
R4 single H1	yes	—
R5 no skipped heading levels	yes	—
R6 index consistency	yes (<code>--all</code>)	—

All six clauses of specification 0.1 are mechanical. Judgment clauses arrive with later versions of the specification; this table is amended with them.

Changelog

- **1.0** (2026-09-19) — Initial: VER-1, VER-2.